K. K. Wagh Education Society's
K. K. Wagh Institute of Engineering Education & Research
Department of Information Technology

GROUP PROJECT REPORT

Submitted in partial fulfilment of the requirements for the degree of
Bachelor of Technology
in
Information Technology

Explainable AI Framework with Explanation Validation for Financial Decision Support

TrustFin AI

Submitted by

Ashwin Patil	Roll No. 46
Aaditya Patil	Roll No. 44
Kanad Nerkar	Roll No. 41
Ved Kalantri	Roll No. 28

Project Guide
Prof. Rupali M. Bora

K. K. Wagh Education Society's
K. K. Wagh Institute of Engineering Education & Research

Academic Year 2026–2027

K. K. Wagh Institute of Engineering Education & Research
Department of Information Technology

CERTIFICATE

This is to certify that the group project report entitled **"Explainable AI Framework with Explanation Validation for Financial Decision Support"**, developed as **TrustFin AI**, is submitted by the following students of the Department of Information Technology in partial fulfilment of the requirements for the degree of Bachelor of Technology:

Ashwin Patil	Roll No. 46
Aaditya Patil	Roll No. 44
Kanad Nerkar	Roll No. 41
Ved Kalantri	Roll No. 28

The project work was carried out under the guidance of **Prof. Rupali M. Bora** during the academic year 2026–2027. Official certification, approval date, and signatures are to be completed by the institute.

Prof. Rupali M. Bora	Head of Department
Project Guide	Director

Date: _____ Place: Nashik

Acknowledgement

The group would like to express its sincere gratitude to the project guide, Prof. Rupali M. Bora, for guidance and feedback during the development of TrustFin AI. The group also thanks the Department of Information Technology and K. K. Wagh Institute of Engineering Education & Research for providing the academic environment in which the project was carried out.

The project brought together work in data preparation, machine learning, explainability, backend services, database persistence, and interface development. The group acknowledges the authors and research communities whose published work on credit scoring, SHAP, LIME, explanation evaluation, and responsible machine learning informed the technical study. All external publications used in this report are cited in the references.

Finally, the group thanks its members for the shared effort involved in designing, implementing, reviewing, and documenting the project. Administrative acknowledgements and any additional institutional wording may be added during final submission review.

Abstract

TrustFin AI is a completed loan decision-support system that combines product-specific credit prediction with post-hoc explanations and quantitative checks on those explanations. The project addresses a practical limitation of using complex models for lending: a prediction can be useful for ranking risk while remaining difficult for an applicant or reviewing officer to interpret. A single explanation is also not sufficient evidence that the explanation is faithful, stable, consistent across methods, or insensitive to a protected characteristic.

The implemented system supports home-loan and personal-loan applications. Each product uses a separately trained XGBoost classifier and a product-specific preprocessing pipeline. Derived repayment features include monthly instalment and fixed-obligation-to-income ratio (FOIR); the home-loan pipeline also derives loan-to-value ratio. The prediction service returns the model decision and class probability, and produces local explanations using TreeSHAP and tabular LIME. An explanation-validation stage calculates faithfulness, stability under a small seeded perturbation, top-feature consistency between SHAP and LIME, and an individual counterfactual gender-flip score. These values are combined using documented weights to form a composite Trust Score. The result, selected applicant fields, explanation data, and trust measures can be stored through SQLAlchemy using local SQLite or a configured PostgreSQL database. A React dashboard presents results to applicant and bank-officer views and the backend can generate a PDF report.

The project datasets are generated by repository scripts and contain 3,000 records each; they are not records from a lending institution. On the stored stratified test evaluations, the home-loan model reports 97.00% accuracy and 0.9978 ROC-AUC, while the personal-loan model reports 97.33% accuracy and 0.9963 ROC-AUC. These values describe performance on generated data and do not establish performance on real applicants. The composite score is an engineering indicator defined by the project, not an independently validated measure of safety or lending quality. The report therefore distinguishes predictive performance, prediction confidence proxy, explanation checks, and real-world fairness evidence.

Keywords: Explainable AI, credit scoring, XGBoost, SHAP, LIME, explanation validation, fairness, Trust Score.

Contents

Certificate **i**

Acknowledgement **ii**

Abstract **iii**

1 Introduction **1**

2 Literature Survey **5**

3 Problem Statement and Requirements **9**

4 System Analysis and Design **13**

5 System Implementation and Methodology **20**

6 Testing and Validation **28**

7 Results and Discussion **30**

8 Project Planning **35**

9 Conclusion and Future Scope **38**

List of Abbreviations

AI	Artificial Intelligence
API	Application Programming Interface
AUC	Area Under the Receiver Operating Characteristic Curve
DFD	Data Flow Diagram
DTI	Debt-to-Income Ratio
EMI	Equated Monthly Instalment
FOIR	Fixed Obligations to Income Ratio
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
LIME	Local Interpretable Model-agnostic Explanations
LTV	Loan-to-Value Ratio
ML	Machine Learning
ORM	Object-Relational Mapper
ROC	Receiver Operating Characteristic
SHAP	SHapley Additive exPlanations
SQL	Structured Query Language
XAI	Explainable Artificial Intelligence
XGBoost	eXtreme Gradient Boosting

List of Figures

4.1 System architecture 13
4.2 Loan evaluation workflow 14
4.3 Evaluation and batch flowchart 14
4.4 Context-level DFD 15
4.5 Level 1 DFD 15
4.6 Applicant and officer use cases 16
4.7 Implemented class diagram 16
4.8 Single-application sequence 17
4.9 User activity diagram 17
4.10 Software component diagram 17
4.11 Local development deployment 18
4.12 Database ER diagram 18
4.13 Database schema 19

7.1 Loan application form 33
7.2 Bank-officer results 33
7.3 Applicant results 34
7.4 Generated PDF preview 34

List of Graphs

5.1 Generated dataset class counts 22
7.1 Local SHAP explanation 32
7.2 Local LIME explanation 32

List of Tables

2.1 Selected literature and its relationship to TrustFin AI 8

3.1 Implemented functional requirements 11

4.1 Principal database entities 19
4.2 Principal API endpoints 19

6.1 Testing and validation evidence 28

7.1 Generated dataset summary 30
7.2 Stored product-specific test metrics 31

8.1 Development phases from Git history 35
8.2 Group members and contribution records 36
8.3 Commit-evidenced activity chart 37

Chapter 1

Introduction

1.1 Introduction

TrustFin AI is a group project that studies how local explanations and explanation checks can be included in a loan decision-support application. The system accepts an applicant profile, selects a home-loan or personal-loan model, returns a binary approval prediction and probability, and generates SHAP and LIME explanations. Four validation values are then calculated and combined into a composite Trust Score. Results are presented through a browser dashboard and can be stored for later review.

The central concern is not simply whether a classifier can produce a high score on a test split. In a consequential setting such as credit assessment, users also need to understand what information influenced an output and whether a generated explanation behaves consistently under defined checks. TrustFin AI therefore treats explanation quality as a measurable part of the workflow. It does not treat the resulting Trust Score as proof that a lending decision is correct, fair, or suitable for deployment.

1.2 Background

Credit scoring uses applicant and account information to estimate the likelihood of a credit outcome. Statistical classification has been used for consumer credit for decades, with studies comparing a range of classification methods and discussing domain-specific problems such as incomplete applicant histories and reject inference [1, 2]. Modern ensemble methods can model non-linear interactions in structured records, but their internal logic is less direct to communicate than a small scorecard or decision tree.

An explanation method attempts to summarize a model's behavior for a particular input. SHAP assigns feature contributions using a Shapley-value framework, while LIME fits an interpretable approximation around an instance. These methods answer related but not identical questions and can disagree. Moreover, an explanation that looks plausible is not necessarily faithful to the model, robust to small input changes, or neutral with respect

to a protected feature. Research has therefore examined explanation quality as a separate evaluation problem rather than assuming that an explanation is valid merely because it was generated [3, 4].

1.3 AI in Financial Decision-Making

Structured credit data are a natural setting for supervised classification: records contain numerical fields such as income, requested amount, repayment term, and credit indicators, together with categorical attributes. TrustFin AI uses XGBoost for its two product-specific models. The model provides a decision and a probability for the approval class. In the application, the probability associated with the selected class is also returned as a confidence representation.

Automated or model-assisted finance requires attention to the complete decision path. Input handling affects what the model receives; preprocessing affects feature scale and representation; a prediction probability is not the same as an explanation; and an explanation is not an audit of model fairness. A useful decision-support interface should preserve these distinctions and show enough information for an officer or applicant to understand the result. General risk-management guidance likewise emphasizes that system risks depend on context and should be considered throughout development and use [5].

1.4 Motivation

The project was motivated by three connected needs. First, applicant-facing systems should communicate the factors associated with a model output in accessible language. Second, a single explainer may not be a sufficient basis for review, so the application should compare SHAP and LIME outputs and report differences. Third, explanation quality should be assessed along more than one dimension: whether influential features affect the model output, whether rankings persist after a small perturbation, whether two methods select similar leading features, and whether a protected-attribute counterfactual changes the probability.

The motivation is deliberately limited. The repository contains generated datasets and experimental model artifacts, not production lending records. TrustFin AI demonstrates an engineering workflow for prediction and explanation review; it is not a validated credit policy or a substitute for institutional model governance and human review.

1.5 Aim of the Project

The completed project delivers a loan decision-support system that produces product-specific predictions, presents local model explanations, and calculates defined checks on the generated explanations before displaying a composite explanation-reliability indicator.

1.6 Objectives

The project objectives are:

1. prepare separate feature pipelines for home-loan and personal-loan records;
2. train and evaluate XGBoost classifiers using fixed train-test partitions;
3. integrate SHAP and LIME explanations for individual prediction requests;
4. calculate faithfulness, stability, explainer consistency, and individual counterfactual fairness scores from the implementation;
5. combine the four values using the implemented weighted formula and return a prediction-confidence representation;
6. expose the workflow through a React interface and FastAPI service, with evaluation persistence and PDF reporting.

1.7 Scope

The implemented scope includes home-loan and personal-loan application forms, their trained models and engineered financial features, single-record prediction, SHAP and LIME explanations, four explanation checks, composite Trust Score, SQLite/PostgreSQL persistence, batch CSV processing, evaluation history, and PDF report generation. The interface provides applicant and bank-officer views; these are presentation modes selected in the client, not authenticated accounts.

The project does not implement a real lender integration, a production underwriting policy, secure user authentication, population-level fairness monitoring, probability calibration, or an independently validated Trust Score threshold. The interactive what-if calculator is a deterministic affordability scenario, not a live call to the prediction model. Fraud detection and unrelated financial products are outside the scope.

1.8 Implemented Approach

The implemented workflow is: collect a product-specific application; validate and normalize request fields; engineer repayment features; transform fields using the product preprocessor; obtain an XGBoost decision and probability; compute SHAP and LIME local explanations; evaluate four explanation properties; calculate the weighted Trust Score; return a structured response; and present, persist, or export the result. This sequence is expanded in Chapters 4 and 5.

1.9 Organization of the Report

Chapter 2 reviews credit scoring, post-hoc explanation methods, explanation evaluation, and fairness concepts. Chapter 3 defines the project problem and requirements. Chapter 4 describes the architecture, data flow, UML views, and database design. Chapter 5 documents the datasets, feature engineering, models, explainers, validation formulas, APIs, interface, persistence, and reporting. Chapter 6 maps the repository's test suite to planned validation and distinguishes designed tests from executed results. Chapter 7 reports stored model metrics and discusses limitations. Chapter 8 summarizes development phases supported by repository history. Chapter 9 concludes and identifies future work.

Chapter 2

Literature Survey

2.1 Introduction

This chapter reviews work relevant to the implemented system: consumer credit classification, gradient-boosted trees, post-hoc local explanation, objective explanation checks, and fairness measures. The focus is on concepts used to motivate or interpret TrustFin AI. The project does not claim to introduce new versions of SHAP, LIME, or the fairness criteria in the literature.

2.2 Machine Learning in Loan Approval

Hand and Henley surveyed statistical classification methods used in consumer credit scoring and described issues specific to credit data, including the limited public availability of lending datasets [1]. Later benchmarking work compared a large set of classifiers across multiple credit-scoring datasets, showing why model choice should be assessed empirically rather than inferred from a single algorithm's reputation [2]. These studies provide context for using supervised learning while also highlighting the importance of dataset properties and evaluation design.

Gradient boosting builds an additive model by fitting successive learners to reduce a loss function. Friedman formalized gradient boosting as a function-approximation procedure [6]. XGBoost provides an optimized tree-boosting implementation with regularization and systems techniques designed for efficient learning on structured data [7]. TrustFin AI uses XGBoost because it is suitable for its tabular, mixed-type loan records and supports the selected TreeSHAP workflow. This engineering choice alone does not establish superiority over other classifiers; no current product-specific comparison experiment is stored in the repository.

2.3 Explainable Artificial Intelligence

Explainable AI includes methods intended to make model behavior more understandable to people. Explanations may describe global patterns, local feature attributions, example-based behavior, or simpler models. TrustFin AI implements local post-hoc explanations for individual loan records. Such explanations should be interpreted as descriptions of model behavior under a chosen method and representation. They do not, by themselves, establish a causal relationship between a feature and repayment or prove that changing a feature would change a real-world outcome.

2.4 SHAP

Lundberg and Lee presented SHAP as a unified framework for additive feature attribution, connecting local explanation methods to Shapley-value principles [8]. In a local additive explanation, the model output is represented through a baseline and feature contributions. The sum of contributions is associated with the explained prediction in the chosen output space. The exact interpretation depends on the explainer, model, and output convention.

TrustFin AI initializes a SHAP TreeExplainer for each loaded XGBoost product model. For a request, the transformed instance is passed to the explainer and returned contributions are converted into a common schema containing feature name, value, contribution, and direction. The dashboard uses these contributions to show leading decision drivers. In this implementation, the explainer operates on preprocessed model features while the display path attempts to retain applicant-level values where names align. This mapping should be reviewed when interpreting categorical one-hot features and engineered columns.

2.5 LIME

Ribeiro, Singh, and Guestrin introduced LIME as a model-agnostic approach that explains a prediction by fitting a simple interpretable model in the neighborhood of the instance [9]. LIME's explanation is local: it depends on the perturbation distribution, similarity weighting, feature representation, and surrogate fitting settings. It is not a global account of the classifier.

TrustFin AI initializes a tabular LIME explainer with a transformed background sample and the model's transformed feature names. At prediction time the model's probability function is passed to LIME. The implementation returns feature contributions in the common explanation schema. Applicant-specific deterministic seeds are used to reduce run-to-run variation in LIME's sampling for identical inputs. Seed control improves repeatability under this implementation; it does not remove sampling approximation or establish correctness of a local surrogate.

2.6 Explanation Validation

Explanation quality is multi-dimensional. A feature ranking can appear stable while being disconnected from model behavior; an explanation can change under small perturbations; and separate methods can select different features. Yeh et al. study measures related to infidelity and sensitivity and show that explanation evaluation involves choices about perturbations and objective functions [3]. Alvarez-Melis and Jaakkola identify faithfulness and stability as important explanation desiderata and examine robustness under changes in input [4].

The TrustFin implementation translates several such concerns into local checks. Its faithfulness score masks selected attribution features and compares changes in model probability against a zero-vector baseline. Stability is measured as a rank correlation between SHAP contributions before and after seeded Gaussian noise in transformed feature space. Consistency is the Jaccard similarity between top-three SHAP and LIME feature sets. These are project-specific operational definitions. Their scores should not be treated as interchangeable with every published fidelity, sensitivity, or agreement metric.

2.7 Trustworthy and Fair AI

Fairness can be defined in different ways, and metrics may express different policy choices. Hardt, Price, and Srebro formalize equality of opportunity in supervised classification in terms of true-positive rates across groups [10]. TrustFin AI's live request metric is different: it compares the model's approval probability for one applicant with the probability after flipping the gender field while holding the remaining normalized profile fixed. That individual counterfactual score is not a population-level disparate-impact or equal-opportunity audit.

The saved training metrics also contain a female-to-male approval-rate ratio calculated on each generated test split. This is a group-level descriptive statistic for the synthetic evaluation data, while the online counterfactual score is per applicant. Neither number on its own establishes that a deployed credit system is fair. The NIST AI Risk Management Framework describes trustworthy-AI characteristics and risk-management activities as context-dependent and lifecycle-oriented [5]; TrustFin AI is a completed academic implementation and has not undergone an institutional AI risk-management assessment.

Counterfactual explanations are another related area. Wachter, Mittelstadt, and Russell discuss explanations framed as changes to facts that could produce a desired result, while noting technical and normative constraints [11]. TrustFin's what-if calculator does not implement a counterfactual search algorithm. It adjusts three financial inputs and recalculates an illustrative affordability ratio; it does not query the classifier or guarantee an outcome.

2.8 Comparative Study of Existing Research

Table 2.1 summarizes the relevance and limits of selected work for this project.

Table 2.1: Selected literature and its relationship to TrustFin AI

Work	Main contribution	Relevance and limitation for this project
Hand and Henley [1]	Review of consumer credit classification methods and domain issues	Establishes credit scoring as a specialized classification problem; does not validate the synthetic TrustFin datasets.
Chen and Guestrin [7]	Scalable regularized tree boosting system	Supports the choice of XGBoost for structured records; does not demonstrate this project's model is best.
Lundberg and Lee [8]	Additive attribution framework with Shapley-based properties	Provides the conceptual basis for TreeSHAP local feature contributions.
Ribeiro et al. [9]	Local surrogate explanations for black-box predictions	Provides the basis for the LIME tabular explanation path; LIME depends on local perturbation choices.
Yeh et al. [3]	Objective analysis of explanation infidelity and sensitivity	Motivates checking explanations separately; TrustFin's calibrated faithfulness score is a custom implementation.
Hardt et al. [10]	Equality-of-opportunity fairness criterion	Clarifies that fairness has multiple definitions; TrustFin's online gender-flip score is not this group criterion.

2.9 Research Gap

The literature provides explanation methods and several proposals for evaluating explanations, but no single scalar is universally accepted as a complete measure of explanation trustworthiness. The project addresses an engineering gap: a working loan-assessment system can show two local explanations and attach several explicit checks to each decision. The contribution is the integration and examination of these checks in a completed system, not a new theoretical metric or evidence of real-world lending validity.

2.10 Summary

SHAP and LIME provide complementary local explanation mechanisms, while explanation-evaluation research motivates checking behavior, stability, and agreement separately. Fairness metrics require a clear definition and a relevant population. These findings shape the report's interpretation: TrustFin AI produces observable model outputs and project-defined validation scores, but the scores remain limited to the definitions, data, and perturbations implemented in the repository.

Chapter 3

Problem Statement and Requirements

3.1 Problem Definition

The project addresses the difficulty of reviewing a loan model output when the classifier is more complex than a short rule list. A bare approval or rejection label gives little indication of which model inputs were influential, and one post-hoc explanation does not show whether the explanation is robust or consistent with another method. The completed system produces an individual loan prediction, generates two explanations, computes defined explanation checks, and presents the resulting information through a web interface.

3.2 Existing System

Conventional credit scoring commonly uses structured application and repayment features with statistical or machine-learning classifiers. A model may return a class and probability, but an applicant-facing or officer-facing workflow also needs data validation, understandable reasons, persistence, and a way to inspect prior evaluations. Post-hoc explainers can contribute to this workflow, but they should not be assumed valid without a stated evaluation procedure.

3.3 Limitations of a Prediction-Only Approach

A prediction-only interface omits important context: whether input values were normalized, which features influenced the local result, whether multiple explanation methods agree, and how stable an explanation is after a defined perturbation. Likewise, presenting one scalar called confidence can be misleading if users interpret it as probability calibration or as trust in the explanation. The system therefore returns separate prediction and explanation-quality fields and labels the composite score as a project-defined indicator.

3.4 Implemented System

TrustFin AI provides two loan products, each with its own XGBoost model, preprocessor, engineered financial features, and explanation background sample. A FastAPI route normalizes an application, calculates monthly EMI and FOIR, gets a model prediction, generates SHAP and LIME explanations, evaluates the four configured checks, creates a dossier identifier, and attempts to persist an evaluation record. The React client presents an applicant view and a bank-officer view. Saved records can be inspected in history, batch CSV records can be processed, and a ReportLab service generates a PDF assessment report.

3.5 Advantages

The system makes the prediction and explanation stages inspectable in one workflow. Product-specific fields prevent property-collateral features from being treated as personal-loan fields and keep revolving-credit and loan-purpose fields specific to unsecured lending. Multiple explanation checks expose more information than a single attribution chart. Local SQLite support allows use without a hosted PostgreSQL service. The record schema preserves input JSON and model outputs for later inspection.

These advantages are bounded by the generated datasets, one stored split per model, local metric definitions, and the absence of server-side authentication. TrustFin AI is a completed academic project; its synthetic-data evaluation and access-control design define its intended operating scope.

3.6 Functional Requirements

Table 3.1 records the functional scope found in the application.

Table 3.1: Implemented functional requirements

ID	Implemented functionality
FR-01	Select a home-loan or personal-loan application flow.
FR-02	Validate and normalize application fields before model inference using Pydantic schemas and product normalization.
FR-03	Generate a product-specific approval prediction and class probability.
FR-04	Generate and return local SHAP and LIME feature contributions.
FR-05	Calculate faithfulness, stability, consistency, and individual counterfactual scores, then combine them into the weighted Trust Score.
FR-06	Display evaluation results in applicant and bank-officer presentation modes.
FR-07	Persist and retrieve evaluation history and batch-upload records through SQLAlchemy.
FR-08	Process product-specific CSV or JSON batches and retain flawed rows with validation reasons.
FR-09	Generate an application PDF report for preview or download.
FR-10	Recalculate illustrative affordability values in the product-specific what-if calculator.

3.7 Non-Functional Requirements

The implementation aims for a modular separation between React presentation, FastAPI routes, model registry, explanation components, validation functions, database access, and PDF reporting. Pydantic schemas provide request and response structure. Reproducibility is supported through a SHA-256-derived applicant seed for LIME sampling and a seeded Gaussian perturbation for the stability check. Local SQLite is configured as a zero-configuration database path, with optional PostgreSQL connectivity. These properties are implemented design choices; no service-level performance, availability, or production security target is measured in the repository.

3.8 Hardware Requirements

The repository does not specify a tested hardware bill of materials. Development requires a computer capable of running Python with the listed scientific packages and a Node.js-based frontend toolchain. Model inference and local SQLite use are intended for a development machine; production capacity, concurrent user limits, and hardware sizing remain unmeasured.

3.9 Software Requirements

The project uses Python, FastAPI, Pydantic, SQLAlchemy, pandas, NumPy, scikit-learn, XGBoost, SHAP, LIME, and ReportLab on the backend. The frontend uses React, Vite, Axios, Tailwind CSS, and Lucide icons. The README documents Python 3.10+ and Node.js 18+ as prerequisites. The API may use local SQLite or a configured PostgreSQL URL. Exact installed package versions depend on the environment because most Python requirements are minimum-version specifications.

3.10 Project Constraints

The main constraints are the synthetic nature of the two product datasets, the limited hold-out evaluation evidence, the dependence of explanations on preprocessing and perturbation assumptions, and the lack of an authenticated access-control layer. The Trust Score is weighted according to one project configuration and has no external calibration study. Population fairness, causal validity, and live lender integration are outside the implemented scope.

Chapter 4

System Analysis and Design

4.1 System Overview

TrustFin AI is organized as a browser client, a REST API, two product pipelines, a database layer, and a PDF reporting function. The client gathers application fields and sends them to FastAPI. The API chooses a pipeline from the requested loan type, normalizes and engineers fields, obtains a classification result, builds two local explanations, calculates validation values, and returns a typed response. The persistence layer stores evaluation and batch data when available. The client is responsible for presentation; it does not implement the ML or explanation algorithms.

4.2 System Architecture

Figure 4.1 shows the principal runtime components. The architecture has one API and model registry but separate home-loan and personal-loan pipelines. Database operation can use local SQLite or a PostgreSQL connection configured through the environment. If the primary configured database is unavailable, the backend session dependency initializes and uses local SQLite. The UI role selector changes presentation only; it is not an authentication boundary.

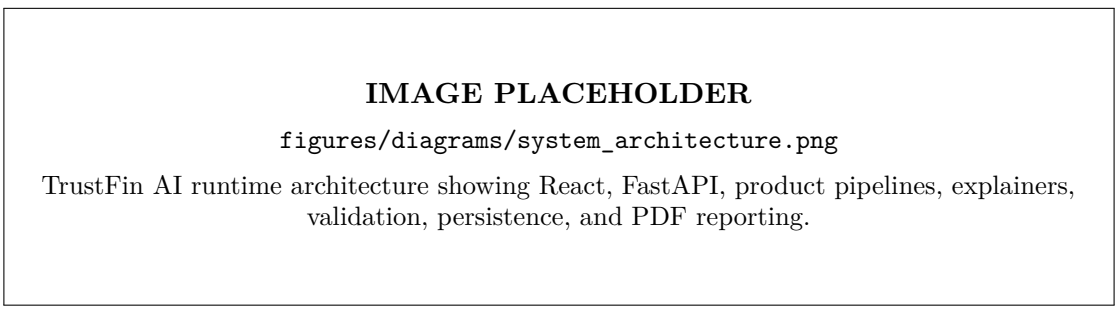


Figure 4.1: TrustFin AI runtime architecture showing React, FastAPI, product pipelines, explainers, validation, persistence, and PDF reporting.

4.3 Overall Workflow

The end-to-end workflow is summarized in Figure 4.2. Inputs are validated through the request schema and product normalization code before feature engineering. For non-legacy products, the selected product feature function derives EMI and FOIR; the home-loan feature function additionally derives LTV. The transformed record is used for inference and explanations. The response contains the raw model decision and solvency summary as separate outputs. In particular, the implementation does not replace an approved model class with a rejection solely because FOIR exceeds 50%; it reports a solvency status and warning alongside the model result.

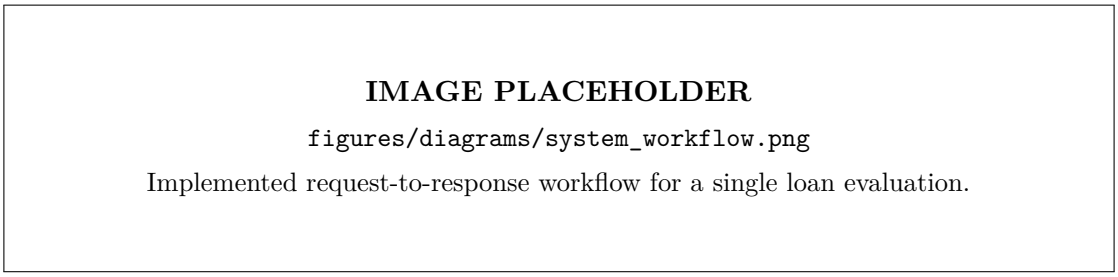


Figure 4.2: Implemented request-to-response workflow for a single loan evaluation.

4.4 Flowchart

The flowchart in Figure 4.3 should distinguish successful validation and model execution from exceptional or flawed-record handling. In single evaluation, an invalid request is rejected through FastAPI/Pydantic validation. In batch evaluation, rows are validated individually; usable records are processed and invalid records are retained as flawed data with row-level reasons. Model prediction, explanation generation, trust calculation, and persistence occur in the route's pipeline.

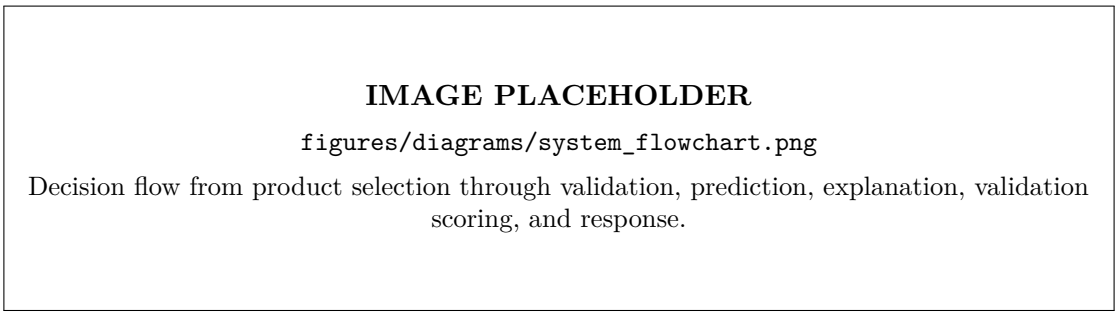


Figure 4.3: Decision flow from product selection through validation, prediction, explanation, validation scoring, and response.

4.5 Data Flow Diagram

The context-level DFD in Figure 4.4 treats the applicant or officer as an external actor who submits an application and receives an assessment. The actor is not represented as a database user because the repository has no authentication table or user service. The system boundary contains the web client and the API-backed model workflow.

IMAGE PLACEHOLDER

`figures/diagrams/context_level_dfd.png`

Context-level data flow between portal users and the TrustFin AI application.

Figure 4.4: Context-level data flow between portal users and the TrustFin AI application.

The Level 1 DFD in Figure 4.5 decomposes the API path into input processing, product feature engineering, inference, explanation generation, validation, persistence, and report generation. Prediction and explanation data are returned to the frontend; persistence is a separate store-and-retrieve path. The database can be local SQLite or configured PostgreSQL.

IMAGE PLACEHOLDER

`figures/diagrams/level_1_dfd.png`

Level 1 data flow for evaluation, storage, history retrieval, batch processing, and PDF output.

Figure 4.5: Level 1 data flow for evaluation, storage, history retrieval, batch processing, and PDF output.

4.6 Use Case Diagram

Figure 4.6 presents the user-visible functions. The applicant can submit an application, inspect a decision, read local explanation and Trust Score information, use an affordability scenario calculator, view the submitted form, and preview or download a PDF. A bank-officer presentation mode can additionally access saved history, batch upload, and audit views. Since there is no server-side authentication, the actor distinction represents UI navigation, not enforced authorization.

IMAGE PLACEHOLDER

`figures/diagrams/use_case_diagram.png`

Use cases for applicant and bank-officer presentation modes.

Figure 4.6: Use cases for applicant and bank-officer presentation modes.

4.7 Class Diagram

The class diagram in Figure 4.7 uses classes that exist in the codebase: Pydantic request/response and explanation schemas, SQLAlchemy record models, the model registry and product pipeline, and SHAP/LIME explainer wrappers. The API route itself is organized as module-level functions rather than a 'PredictionController' class. The database contains no 'User' entity. Logical labels from the project naming convention are used only where they map to implemented modules or schema types.

IMAGE PLACEHOLDER

`figures/diagrams/class_diagram.png`

Implemented schema, pipeline, explainer, and persistence classes with their principal dependencies.

Figure 4.7: Implemented schema, pipeline, explainer, and persistence classes with their principal dependencies.

4.8 Sequence Diagram

Figure 4.8 describes the main single-record request. The client calls the prediction endpoint; the route selects a product pipeline, normalizes input, derives features, and invokes the model and explainers. Validation functions calculate four scores and the Trust Score engine combines them. The response is persisted when the database session is available and returned to the client. The diagram should show the solvency check as reported metadata rather than an independent class override.

IMAGE PLACEHOLDER

`figures/diagrams/sequence_diagram.png`

Sequence of calls for an individual application from React submission through API response.

Figure 4.8: Sequence of calls for an individual application from React submission through API response.

4.9 Activity Diagram

The activity diagram in Figure 4.9 focuses on user-visible paths: choosing a portal mode, selecting a loan product, entering or uploading an application, viewing the assessment, and optionally inspecting history or a PDF. This differs from the model workflow by showing user actions and navigation rather than service calls.

IMAGE PLACEHOLDER

`figures/diagrams/activity_diagram.png`

User activity paths for home-loan and personal-loan evaluations.

Figure 4.9: User activity paths for home-loan and personal-loan evaluations.

4.10 Component Diagram

Figure 4.10 groups code by responsibility. Frontend components consume the API; route functions coordinate preprocessing, product registry, explanations, validation, persistence, and PDF generation. The diagram should not imply a separate authentication service, a portfolio analytics engine, or a deployed monitoring subsystem because those are not implemented.

IMAGE PLACEHOLDER

`figures/diagrams/component_diagram.png`

Software components and their principal dependencies.

Figure 4.10: Software components and their principal dependencies.

4.11 Deployment Diagram

The documented local setup runs a Vite development server and a FastAPI server as separate processes. The API loads model artifacts from the repository and connects to local SQLite by default or to a configured PostgreSQL service. Figure 4.11 represents this development deployment; it is not a production topology or a claim that cloud hosting is configured.

IMAGE PLACEHOLDER

`figures/diagrams/deployment_diagram.png`

Local development deployment with optional PostgreSQL and SQLite fallback.

Figure 4.11: Local development deployment with optional PostgreSQL and SQLite fallback.

4.12 Entity Relationship Diagram

The persistence schema has two principal tables. `PredictionLog` stores the dossier identifier, input JSON, prediction and explanation results, Trust Score fields, and optional batch identifier. `BatchUpload` stores the upload identifier, product, filename, timestamp, counts, flawed-row JSON, and aggregate summary. A batch can be associated with many prediction rows by the shared `batch_id` value, but the source schema does not declare a relational foreign-key constraint between these models. Figure 4.12 shows this as an application-level association.

IMAGE PLACEHOLDER

`figures/diagrams/er_diagram.png`

Logical relationship between batch uploads and stored prediction logs.

Figure 4.12: Logical relationship between batch uploads and stored prediction logs.

4.13 Database Design

Table 4.1 summarizes persisted entities. JSON columns retain full input and explanation structures while scalar columns support history filtering and summary display. The models store no credentials, user roles, or signed officer actions. Schema initialization uses SQLAlchemy metadata creation and a small set of additive column checks; the project does not use a versioned migration framework.

Table 4.1: Principal database entities

Entity	Key fields	Purpose
`PredictionLog`	`id`, unique `dossier_id`	Stores product type, selected applicant fields, raw input JSON, prediction, probability, confidence representation, solvency values, SHAP/LIME JSON, four validation scores, Trust Score, full response, and optional batch ID.
`BatchUpload`	`id`, unique `batch_id`	Stores product and file metadata, timestamp, record counts, flawed records, and a batch summary.

IMAGE PLACEHOLDER

`figures/diagrams/database_schema.png`

Stored fields and JSON payload boundaries for the SQLAlchemy models.

Figure 4.13: Stored fields and JSON payload boundaries for the SQLAlchemy models.

4.14 API and Module Design

The API router uses the prefix `/api/v1/loan`. Table 4.2 lists the principal implemented endpoints. The backend uses typed Pydantic request and response models for application prediction, batch result, evaluation summary, and PDF generation. Detailed response fields are described in Chapter 5.

Table 4.2: Principal API endpoints

Method	Path	Purpose
POST	`/predict`	Evaluate one application and return prediction, explanations, checks, and dossier reference.
POST	`/predict-batch`	Upload and evaluate a CSV file.
POST	`/predict-batch-json`	Evaluate a JSON list of batch records.
GET	`/evaluations`	List recent saved evaluations.
GET	`/evaluations/{id}`	Retrieve one saved evaluation.
DELETE	`/evaluations/{id}`	Delete one saved evaluation.
GET	`/batches`, `/batches/{id}`	List batch uploads and retrieve batch details.
DELETE	`/batches/{id}`	Delete a batch and linked logs.
GET	`/batch-sample-csv`, `/batch-presets/{id}`	Download product-specific sample data or retrieve a preset.
POST	`/generate-pdf`	Generate an application report PDF.

Chapter 5

System Implementation and Methodology

5.1 Technology Stack

The backend is implemented in Python using FastAPI, Pydantic, SQLAlchemy, pandas, NumPy, scikit-learn, XGBoost, SHAP, LIME, and ReportLab. The frontend is a React application built with Vite and styled with Tailwind CSS. Axios is used for API calls. The main application is organized around one loan-prediction router and a registry that loads the home-loan, personal-loan, and legacy model pipelines. The deployed product paths are the home-loan and personal-loan pipelines; the legacy pipeline remains for backwards compatibility.

5.2 Development Environment

The repository README documents Python 3.10 or later, Node.js 18 or later, and separate backend and frontend development servers. The API is typically served by Uvicorn on port 8000 and Vite serves the client during development. Dependencies are described in `requirements.txt` and `frontend/package.json`. The Python requirements use lower version bounds rather than a fully frozen environment, so exact reproducibility also depends on the installed package versions and platform.

5.3 Dataset Description

Two product datasets are present: `datasets/homeloan_train.csv` and `datasets/personalloan_train.csv`. Repository scripts generate each dataset with 3,000 records using NumPy random seed 42. The home-loan table has 18 columns and the personal-loan table has 17. Both contain a loan ID, applicant attributes, financial inputs, credit information, and a target label named `Loan_Status`. The home-loan dataset has 1,529 approved and 1,471 rejected

records. The personal-loan dataset has 1,563 approved and 1,437 rejected records. The checked CSVs have no missing cells and no duplicate full rows.

These are synthetic data generated by the repository. Their label-generation scripts construct the outcome from hand-specified financial and credit rules and include a gender-dependent adjustment in a borderline region to create a fairness-analysis example. The high test scores reported later are consequently measurements on a designed simulation, not evidence of performance on a bank's historical or future applicant population. There is no externally sourced lending dataset or data dictionary in the repository.

5.4 Data Preprocessing

The training scripts read a product CSV, compute derived features, separate `Loan_Status` from the predictors, and drop `Loan_ID`. The target is mapped to 1 for Approved and 0 for Rejected. An 80/20 stratified train-test split is created with random state 42. The preprocessor uses a column transformer: categorical fields pass through a one-hot encoder with the first category dropped and unknown values ignored; numerical fields pass through standard scaling. The transformer is fit on the training set and then applied to the test set.

For online requests, Pydantic schemas normalize optional numeric values and the model registry fills product-specific defaults. This normalization is operationally convenient but means omitted values can be replaced with defaults; users should provide valid application data and review the displayed submitted form. Batch processing applies row-level normalization and records rows that cannot be used as flawed records with explanations of the validation issue.

5.5 Exploratory Data Analysis

No reproducible exploratory-data-analysis notebook or EDA report is included in the current source tree. The dataset-generation scripts specify distributions for demographics, income, loan terms, property characteristics, CIBIL scores, utilization, and missed payments. This section therefore describes those generator rules and the observed row/label counts rather than claiming a separate EDA study. A manually produced chart of class counts may be added at `figures/graphs/dataset_class_distribution.png` as shown in Graph 5.1.

5.6 Feature Engineering

The two products use distinct financial calculations. For a loan principal P, annual interest rate i, and term n months, let $r = i/12$. The amortized monthly instalment used by the product pipelines is

$$\text{EMI} = P\frac{r(1+r)^n}{(1+r)^n - 1}. \tag{5.1}$$

IMAGE PLACEHOLDER

`figures/graphs/dataset_class_distribution.png`

Approved and rejected counts in the two generated product datasets; use the repository CSV counts reported in this chapter.

Graph 5.1: Approved and rejected counts in the two generated product datasets; use the repository CSV counts reported in this chapter.

The home-loan generator and preprocessing function use an illustrative annual rate of 8.5%; the personal-loan path uses 13.5%. These fixed rates are implementation assumptions for engineered features, not live lender quotations.

For a home loan, monthly household income is the sum of applicant and co-applicant income. For a personal loan, the FOIR denominator is applicant income. In both product-specific pipelines the ratio is

$$\text{FOIR} = \frac{\text{Existing EMI} + \text{Proposed EMI}}{\text{Income used for product}}. \tag{5.2}$$

The home-loan pipeline also calculates

$$\text{LTV Ratio} = 100 \times \frac{\text{Loan Amount}}{\text{Property Value}}. \tag{5.3}$$

Personal-loan features include loan purpose and credit-card utilization instead of property collateral features. Home-loan raw fields include the property value and area; personal-loan raw fields include purpose and utilization. Engineered EMI/FOIR and product-specific columns are included in each model's numerical feature set.

5.7 Machine Learning Model Development

Separate XGBoost classifiers are trained in `scripts/train_homeloan.py` and `scripts/train_personalloan.py`. Both scripts use 120 estimators, maximum tree depth 4, learning rate 0.08, row subsampling 0.85, column subsampling 0.85, random state 42, and log-loss as the evaluation metric. Each script saves the classifier, fitted preprocessor, transformed feature names, a 100-row transformed background sample, and metrics under the product directory. The runtime registry loads those artifacts into a `ProductPipeline` with the product feature function and explainers.

Algorithm 1: Product-specific model training

```
Input: Product dataset D, product feature function g, seed 42
Output: Fitted preprocessor T, XGBoost classifier M, test metrics
```

```
1. D' <- g(D); remove Loan_ID from predictors
2. Encode Loan_Status: Approved=1, Rejected=0
3. Split predictors and labels with stratification (80% / 20%)
4. Fit one-hot encoding and standard scaling on training predictors
5. Transform training and test predictors with the fitted
   transformer
6. Fit XGBoost(n_estimators=120, max_depth=4, learning_rate=0.08,
   subsample=0.85, colsample_bytree=0.85, random_state=42)
7. Predict labels and approval probabilities on the held-out test
   split
8. Save model, transformer, feature names, background sample, and
   metrics
```

5.8 Model Training and Evaluation

The training scripts calculate accuracy, precision, recall, F1-score, and ROC-AUC on the held-out split. Product-specific metrics are saved in `models/homeloan/metrics.json` and `models/personalloan/metrics.json`. No product-specific confusion matrix is stored in those files. The root `models/model_metadata.json` describes an older legacy model and must not be substituted for the current product metrics. Chapter 7 reports the product-specific artifacts and keeps the legacy results separate.

5.9 SHAP Implementation

The model registry initializes a `SHAPExplainer` backed by `shap.TreeExplainer` for each XGBoost classifier. For each application the transformed feature vector is passed to the explainer. The implementation extracts the contribution vector, maps the sign to positive, negative, or neutral direction, and returns one feature record for each transformed model feature. Local contribution values are in the SHAP output space used by the explainer; they are not percentages of approval probability. Displaying raw values for one-hot and engineered features requires care because the feature name may not match a raw applicant field.

5.10 LIME Implementation

The `LIMEExplainer` wraps `lime.lime_tabular.LimeTabularExplainer` in classification mode. The explainer receives the transformed training background and model feature names. When explaining an input, it calls the classifier's `predict_proba` function and requests contributions for the approved class. The route sets a deterministic seed derived from the input JSON using SHA-256 before running LIME. This supports repeatable sampling for

the same input in a fixed software environment; it does not imply that all library versions or hardware produce bitwise identical results.

5.11 Explanation Validation Framework

The route applies four checks after generating the local explanation. The formulas below reproduce the code-level definitions and are not presented as universal standards.

5.11.1 Faithfulness

The implementation uses the SHAP explanation for the current model prediction. For an approved prediction, it selects positive SHAP contributors above 0.08, falling back to up to five largest positive contributions. For a rejected prediction, it selects negative contributors below -0.08, falling back to up to five largest negative contributions. The selected transformed coordinates are replaced with zero. Let p be the original approval probability, p_m the probability after masking, and p_0 the probability for the zero-vector baseline. Define

$$\Delta = |p - p_m|, \qquad q = \frac{\Delta}{\max(0.08, |p - p_0|)}. \tag{5.4}$$

The implemented score is

$$F = \begin{cases} \max(0.02, 2q), & q \leq 0.05, \\ \min(0.95, 0.74 + 0.17(1 - e^{-q})), & q > 0.05. \end{cases} \tag{5.5}$$

If no salient features are selected, the function returns 0.10. The piecewise mapping is a project calibration heuristic. Its numerical scale is not a conventional probability of explanation correctness and should not be compared directly with unrelated faithfulness measures.

5.11.2 Stability

The route perturbs the transformed input with a Gaussian random vector of standard deviation 0.01 using a seed derived from the applicant input. The original and perturbed SHAP contribution vectors are compared with Spearman rank correlation ρ. The returned score is

$$S = \frac{\rho + 1}{2}, \tag{5.6}$$

which maps the correlation range from $[-1, 1]$ to $[0, 1]$. If the correlation is undefined because contributions are constant, the implementation returns zero. This measures rank similarity under the specific perturbation procedure; it is not a guarantee of invariance under all plausible applicant changes.

5.11.3 Consistency

Let A_3 and B_3 be the sets of three features with largest absolute contributions under SHAP and LIME. The consistency score is their Jaccard similarity:

$$C = \frac{|A_3 \cap B_3|}{|A_3 \cup B_3|}. \tag{5.7}$$

The code compares feature names only; it does not compare contribution magnitudes or directions.

5.11.4 Individual Counterfactual Fairness Score

For a normalized applicant profile x, the code obtains approval probabilities $p(x)$ and $p(x^{(g)})$, where $x^{(g)}$ has the gender field flipped and other fields held fixed. The score is

$$R = \max\left(0, \min\left(1, 1 - |p(x) - p(x^{(g)})|\right)\right). \tag{5.8}$$

This is an individual model-sensitivity check for one binary attribute. It is not an estimate of group approval-rate parity, equal opportunity, causal fairness, or legal compliance.

5.12 Trust Score Calculation

The Trust Score is the weighted sum below, with all component scores on the interval $[0, 1]$:

$$T = 0.35F + 0.25S + 0.30C + 0.10R. \tag{5.9}$$

The weights are stored in `backend/app/trust/trust_score.py`. Faithfulness receives 35%, stability 25%, SHAP-LIME consistency 30%, and the counterfactual gender-flip score 10%. The result is a project-defined summary used for presentation. No external calibration study or validated decision threshold for this composite is included.

5.13 Confidence Score

Let p be the model's probability for approval and $\hat{y}$ its predicted class. The response confidence value is

$$Q = \begin{cases} p, & \hat{y} = 1, \\ 1 - p, & \hat{y} = 0. \end{cases} \tag{5.10}$$

It is the model probability assigned to the predicted class. The repository does not run a separate calibration method such as isotonic regression or Platt scaling for these product models. Therefore the value should be called a prediction-confidence representation, not a verified calibration estimate or an explanation-reliability measure.

5.14 Backend Implementation

The FastAPI router is mounted under `/api/v1/loan`. `POST /predict` performs a single evaluation. Batch endpoints accept CSV uploads or JSON rows. Other endpoints list, retrieve, and delete evaluation or batch history, return sample CSVs and presets, and generate a PDF report. Pydantic schemas define application fields, optional product fields, explanation structures, solvency data, and response objects.

5.15 Frontend Implementation

The React application uses a screen state to select a portal mode, loan product, application form, and result view. Bank-officer and applicant screens are different presentations over shared evaluation results. The dashboard renders prediction information, SHAP/LIME summaries, validation metrics, submitted form values, and a loan-specific what-if affordability calculator. The what-if calculator recomputes EMI and FOIR with fixed illustrative rates; it does not call the model on slider changes. It should therefore not be read as a simulated approval prediction.

5.16 Database Implementation

SQLAlchemy models `PredictionLog` and `BatchUpload` persist evaluation and batch information. Local SQLite is always initialized as a fallback; a PostgreSQL URL can be supplied through `DATABASE_URL`. The request session attempts the primary engine and falls back to SQLite after a connection failure. Database values include JSON fields for raw inputs, full responses, explanations, and flawed batch data. This makes the project runnable without a remote database, although local SQLite and cloud PostgreSQL are operational alternatives rather than a replicated multi-database consistency design.

5.17 API Communication

The frontend sends JSON requests for single predictions and PDF creation, and multipart form data for CSV uploads. The response includes prediction, approval probability, predicted-class confidence representation, model version, SHAP and LIME feature arrays, Trust Score components, solvency summary, and dossier ID. The API's structured response makes each output separately available to the UI and stored history.

5.18 Authentication and Security

The system does not implement account creation, passwords, tokens, OAuth, or server-side role authorization. Applicant and bank-officer selections change client-side navigation and visible controls. The backend CORS middleware allows all origins and credentials in the current configuration. These security properties define the system's deployment limits when handling personal financial data. The project does not claim security certification or regulatory compliance.

5.19 PDF Report Generation

The reporting module uses ReportLab to create an A4 PDF with application reference, applicant and facility summary, decision rationale, solvency information, trust and validation values, and next steps. The PDF endpoint receives applicant data and the prediction response and returns a PDF response. The report is generated on demand; the code does not create a cryptographic document hash or a signed officer approval record.

5.20 End-to-End System Workflow

Algorithm 2: Online loan evaluation

```
Input: LoanApplicationRequest x
Output: prediction response y
1. Select product pipeline M from x.loan_type
2. Normalize product-specific values and construct one-row data
   frame
3. Derive EMI, FOIR, and (for home loans) LTV features
4. Transform the row with the product's fitted preprocessor
5. Obtain model class y_hat and approval probability p
6. Generate SHAP and LIME local explanations
7. Calculate faithfulness, stability, consistency, and gender-flip
   scores
8. Calculate weighted Trust Score and predicted-class probability
   value
9. Generate product-prefixed dossier ID
10. Persist record when database session is available; return
   response
```

This sequence documents the behavior in the API route. It also shows why an explanation-quality score should remain distinct from the prediction probability and the deterministic FOIR summary.

Chapter 6

Testing and Validation

6.1 Overview

Verification evidence for TrustFin AI consists of automated test modules included with the application and saved model-evaluation metrics produced by the training scripts. The software tests cover selected backend and frontend behavior. The model metrics describe classification performance on product-specific stratified hold-out data. These forms of evidence are reported separately because they address different properties of the system.

6.2 Current Testing Summary

Table 6.1 summarizes the verification evidence present in the project.

Table 6.1: Testing and validation evidence

Area	Current evidence in the project
Backend software tests	Pytest modules cover deterministic explanation behavior, data normalization and solvency summaries, batch validation, multi-product prediction endpoints, database persistence and fallback, PDF reporting, and integrated API flows.
Frontend software tests	Vitest modules cover the API client, applicant form, audit-history view, and batch-upload interface.
Model evaluation	Product-specific training artifacts store accuracy, precision, recall, F1-score, ROC-AUC, and a gender approval-rate ratio for the stratified hold-out split. Numerical results are presented in Chapter 7.
Explanation validation	Each prediction response includes the four implemented explanation checks and the weighted Trust Score. Their definitions and interpretation are described in Chapter 5.

6.3 Software Test Coverage

The backend suite is organized by feature area, including audit reproducibility, batch processing, database persistence, product-specific APIs, data and solvency calculations, explainability, reporting, and full-system integration. The frontend suite is organized around the API client and selected form, history, and batch components. This structure connects automated checks to the main application paths without treating the presence of test modules as a model-performance result.

6.4 Model Validation

The home-loan and personal-loan training scripts use an 80/20 stratified train-test split with random state 42. Each saved metric file reports results for 600 held-out records. The metrics describe model behavior on the generated datasets; they do not measure application correctness or establish performance on records from a lending institution. Chapter 7 presents the recorded values and their limitations.

6.5 Explanation Validation

For each online evaluation, the backend computes faithfulness, stability, consistency, and an individual counterfactual gender-flip score, then combines the values using the configured weights. These checks assess defined aspects of the generated local explanations. They do not establish that a loan prediction is correct, calibrated, causally justified, or fair across a population.

6.6 Summary

The repository contains automated backend and frontend test coverage for selected system behavior, while the trained product models have saved hold-out evaluation metrics. The explanation-validation values are generated for individual requests. The report keeps these forms of evidence distinct and does not infer system-test outcomes from model scores.

Chapter 7

Results and Discussion

7.1 Experimental Setup

The product training scripts use 3,000 generated records per product, an 80/20 stratified train-test split, and random state 42. The product-specific metric files report performance on the held-out split. Metrics are included as stored results; this report did not retrain either model or execute new model evaluation. No repeated cross-validation scores or confidence intervals for these product models are stored in the repository.

7.2 Dataset Characteristics

Table 7.1 summarizes the checked datasets. The home-loan dataset contains property value and area, co-applicant income, repayment term, CIBIL score, missed payments, and existing EMI. The personal-loan dataset replaces property-related variables with loan purpose and credit-card utilization. Both datasets have near-balanced labels. The generator scripts use predefined distributions and label rules; consequently, the class balance and observed results reflect those construction choices.

Table 7.1: Generated dataset summary

Product	Records	Input columns including ID/target	Approved	Rejected
Home loan	3,000	18	1,529	1,471
Personal loan	3,000	17	1,563	1,437

The datasets contain no missing cells or duplicate complete rows in the repository snapshot examined. A later dataset regeneration can change those counts if generator logic or random seed changes. Figure 5.1 in Chapter 5 is reserved for a plot of these counts.

7.3 Model Performance

Table 7.2 reports the product metrics saved under `models/homeloan/metrics.json` and `models/personalloan/metrics.json`. Both model scripts calculate metrics on the same 20% stratified held-out split. The home-loan model reports 0.9700 accuracy and 0.9978 ROC-AUC. The personal-loan model reports 0.9733 accuracy and 0.9963 ROC-AUC. Precision, recall, and F1 are also high on these generated test sets.

Table 7.2: Stored product-specific test metrics

Product	Test size	Accuracy	Precision	Recall	F1-score	ROC-AUC
Home loan	600	0.9700	0.9615	0.9804	0.9709	0.9978
Personal loan	600	0.9733	0.9744	0.9744	0.9744	0.9963

The high values are consistent with the explicit, learnable relationships used to generate labels. They should not be read as an expected performance estimate for real loan applications. Data shift, historical lending bias, label quality, reject inference, missingness, and policy differences are not evaluated with external records. Product-specific confusion matrices are not saved in the corresponding metrics files, so this report does not present a reconstructed confusion matrix.

7.4 Evaluation Metrics

Accuracy measures the fraction of correct class predictions. Precision measures the fraction of approved predictions that are correct, and recall measures the fraction of actual approved records identified by the classifier. F1 is the harmonic mean of precision and recall. ROC-AUC measures ranking discrimination over thresholds. Since this is a credit decision-support system, none of these measures alone captures financial harm, calibration, operational policy, or fairness. The dataset construction and class decision threshold must be considered alongside the reported values.

7.5 Group Approval-Rate Ratios

The training scripts also calculate the ratio of the test-set female approval rate to the male approval rate. The stored values are 0.9058 for home loans and 0.8648 for personal loans. These are descriptive ratios on synthetic test records using the model's predicted approvals. The repository does not document a legal threshold for these values, and they do not replace the per-applicant gender-flip score used by the online Trust Score. They should not be presented as proof of fairness.

7.6 SHAP and LIME Results

The application generates local SHAP and LIME feature contributions for each evaluated profile. Contributions can be inspected through the dashboard's feature views and are returned in the API response. The repository does not include a saved aggregate explanation dataset or a fixed experiment report from which to calculate average attribution values across a test cohort. The placeholders in Graphs 7.1 and 7.2 are reserved for user-supplied exports from actual evaluations; they are not fabricated figures.

IMAGE PLACEHOLDER

`figures/graphs/shap_local_explanation.png`

SHAP explanation exported from a real TrustFin evaluation, with feature labels and contribution scale visible.

Graph 7.1: SHAP explanation exported from a real TrustFin evaluation, with feature labels and contribution scale visible.

IMAGE PLACEHOLDER

`figures/graphs/lime_local_explanation.png`

LIME explanation exported for the same evaluation profile used in the SHAP figure.

Graph 7.2: LIME explanation exported for the same evaluation profile used in the SHAP figure.

7.7 Explanation Validation Results

Faithfulness, stability, consistency, and the individual counterfactual score are calculated for an application response. TrustFin AI then computes the weighted Trust Score as documented in Chapter 5. No aggregate test-set distribution, mean score, variance, or benchmark comparison is persisted in the repository artifacts. The Trust Score values visible in individual history records depend on the applications evaluated and are not used here because they are not a controlled, de-identified experimental dataset.

The implementation has useful limitations to keep in view. The faithfulness mapping includes fixed thresholds and a calibrated score band; stability perturbs standardized features with one Gaussian scale; consistency compares only the top three names and not their signs; and the fairness score flips one attribute for one record. These measures are complementary diagnostics, but combining them does not remove their individual assumptions.

7.8 Trust Score and Confidence Score Analysis

The Trust Score measures the weighted combination of the four project-defined explanation checks. Prediction confidence represents the probability assigned to the predicted class. A high prediction confidence does not imply high explanation reliability, and a high Trust Score does not imply that the model's prediction is correct or fair. The application's result screen should display these measures as distinct fields. Product metric artifacts do not report calibration error, Brier score, or reliability diagrams, so probability calibration cannot be assessed from the available outputs.

7.9 GUI and Dashboard Results

The React interface contains separate home-loan and personal-loan flows, an applicant presentation, a bank-officer view, local explanation displays, saved history, CSV batch processing, submitted-form inspection, PDF actions, and a product-specific affordability calculator. Figures 7.1–7.4 are placeholders for screenshots captured from the actual running application. No screenshots are embedded until supplied, as requested.

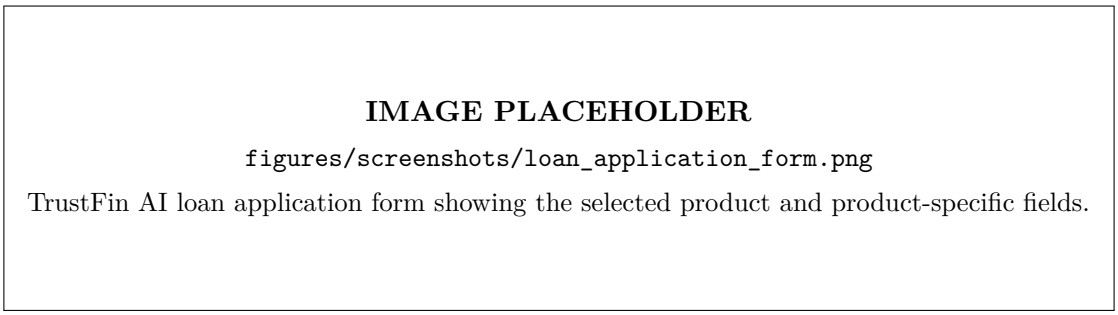


Figure 7.1: TrustFin AI loan application form showing the selected product and product-specific fields.

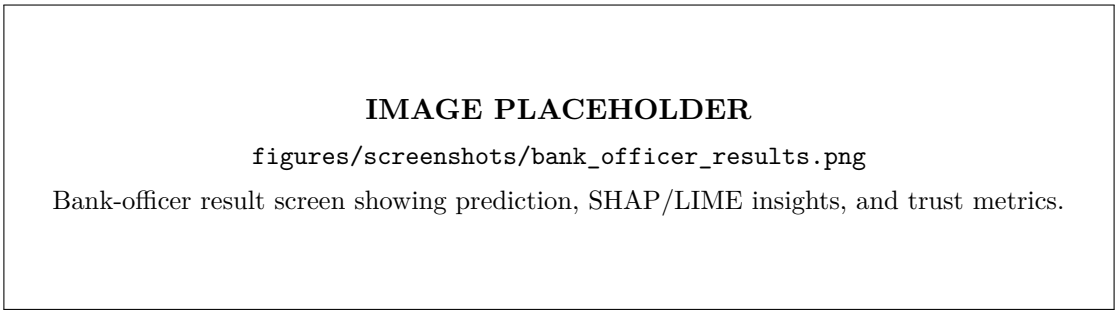


Figure 7.2: Bank-officer result screen showing prediction, SHAP/LIME insights, and trust metrics.

IMAGE PLACEHOLDER

`figures/screenshots/applicant_results.png`

Applicant result screen with submitted-application view and what-if calculator.

Figure 7.3: Applicant result screen with submitted-application view and what-if calculator.

IMAGE PLACEHOLDER

`figures/screenshots/pdf_report_preview.png`

PDF report preview generated by the backend for a real evaluation.

Figure 7.4: PDF report preview generated by the backend for a real evaluation.

7.10 Discussion of Findings

The stored results show that the training scripts and product artifacts achieve high performance on their generated hold-out records. The completed system connects prediction, explanation, validation, interface, storage, and reporting paths. The evidence supports an inspectable loan-assessment workflow on generated data; it does not establish production-level accuracy, regulatory compliance, calibrated confidence, population-level fairness, causal explanation, or external validation of the Trust Score.

Chapter 8

Project Planning

8.1 Development Methodology

The repository history indicates iterative development: early model and interface work was followed by review changes, persistence and batch support, multi-product work, explanation-score refinement, and usability fixes. The available history does not contain a formal sprint plan or approved Gantt baseline. Table 8.1 summarizes date windows supported by commit records; the table is an activity chronology rather than a claim about continuous daily effort.

Table 8.1: Development phases evidenced by repository history

Recorded window	Evidence in history	Phase summary
8–11 Sep 2026	Initial project, deterministic XAI, UI and roadmap commits	Initial system architecture, explanation and trust logic, and interface implementation.
29 Sep 2026	Assessment wording and audit explanation update	Refinement of visible assessment language and audit formulas.
17 Oct 2026	UI simplification, tests, sizing analysis and presentation assets	Interface revisions and test infrastructure additions.
22 Oct 2026	Evaluation history, batch CSV, schema initialization, API tests	Persistence, batch processing, and backend/frontend test modules.
24 Oct 2026	Documentation update commit	Project documentation revisions.
30 Oct–1 Nov 2026	Product-specific loan, Trust Score, SQLite, PDF, calculator, and UI commits	Multi-product refinements, local database support, report and calculator changes, and final navigation improvements.

8.2 Project Timeline and Milestones

The available Git history begins on 8 September 2026 and includes project changes through 1 November 2026. The named milestones in Table 8.1 are based on commit dates and subjects. They do not measure effort or prove that work was finished on the commit date. A formal week-by-week Gantt schedule was not located in the repository.

8.3 Team Responsibilities

The official group contains Ashwin Patil (Roll No. 46), Aaditya Patil (Roll No. 44), Kanad Nerkar (Roll No. 41), and Ved Kalantri (Roll No. 28). Repository history is not sufficient to assign all engineering work to individuals. The responsibility cells are intentionally left for confirmation by the group instead of inferring contribution from commit authors.

Table 8.2: Group members and contribution records to confirm

Member	Roll number	Project responsibility / contribution
Ashwin Patil	46	*To be confirmed by the group*
Aaditya Patil	44	*To be confirmed by the group*
Kanad Nerkar	41	*To be confirmed by the group*
Ved Kalantri	28	*To be confirmed by the group*

8.4 Challenges

Implementation challenges reflected in the code include keeping two product feature sets and preprocessing pipelines aligned, mapping explainers to transformed model features, making stochastic LIME output reproducible for repeated input, maintaining local database operation when PostgreSQL is not available, and separating prediction results from explanation-quality indicators. The code and tests also expose a point requiring review: some test wording describes an automatic solvency rejection while the route currently returns model prediction and solvency assessment as separate results.

8.5 Gantt Chart

The evidence supports milestone dates but not a full task-duration schedule. The following compact project chronology is the Gantt-style view that can be supported without inventing work durations.

Table 8.3: Commit-evidenced activity chart

Activity	Sep 8–11	Sep 29	Oct 17	Oct 22	Oct 24	Oct 30–Nov 1
Initial architecture, XAI and UI	•					
Assessment/audit wording		•				
Testing and UI refinements			•			
Database, history and batch upload				•		
Documentation changes					•	
Multi-product and final refinements						•

Chapter 9

Conclusion and Future Scope

9.1 Conclusion

TrustFin AI implements a loan decision-support workflow that links home-loan and personal-loan models with SHAP and LIME explanations, four explanation checks, a weighted Trust Score, a React dashboard, database persistence, batch evaluation, and PDF reporting. The product models use generated synthetic records and show high stored metrics on stratified hold-out splits. These results are useful for demonstrating the system workflow, but they do not establish real-world underwriting performance.

The project makes an engineering contribution by presenting the prediction and explanation checks as separate outputs within one application. It also makes several assumptions visible: the faithfulness score is heuristically calibrated, stability depends on a specified perturbation, consistency compares a top-three feature set, and fairness is measured through an individual gender-flip probability difference. The confidence value is the predicted-class probability and is not independently calibrated. These distinctions support an accurate interpretation of the completed system.

9.2 Project Contributions

The implemented contributions are:

- two product-specific feature and preprocessing pipelines for home and personal loans;
- XGBoost training and saved test metrics for generated datasets;
- local TreeSHAP and tabular LIME explanation paths;
- code-level metrics for faithfulness, stability, top-feature consistency, and individual counterfactual sensitivity;
- a weighted Trust Score and a separate predicted-class probability field;
- a web interface, evaluation and batch history, local SQLite fallback, and on-demand PDF reporting.

9.3 Limitations

The principal limitations are synthetic-only data, limited external validation, no probability calibration, no group-level fairness audit in the request path, no causal explanation guarantees, and a custom composite score without external validation. The application does not implement authentication or server-enforced role access. The what-if calculator changes affordability inputs but does not request a fresh model prediction. The project also has no stored product-specific confusion matrices or held-out aggregate Trust Score study.

9.4 Future Scope

Future work should first evaluate the models on a lawful, representative, de-identified lending dataset and establish leakage controls, missing-data policy, calibration, subgroup metrics, and temporal validation. Explanation metrics should be tested against independently justified perturbation schemes and human review, with sensitivity analyses for thresholds and weights. The Trust Score should be treated as a hypothesis until an evaluation study demonstrates how it relates to meaningful explanation quality or reviewer decisions.

The application could add authentication, server-side authorization, privacy controls, audit retention policies, versioned database migrations, secure deployment configuration, monitoring, and clear human-review workflows. The what-if calculator could call a controlled scenario endpoint if the intended behavior is to recompute model output; such a change would require displaying the changed inputs and preventing the scenario from being mistaken for an approved loan. Additional explainers or constrained counterfactual methods may be evaluated as future research, not described as existing functions.

9.5 Closing Statement

The completed implementation brings loan prediction, local explanation, explanation validation, history, batch processing, and reporting into one decision-support system. Its contribution is the integration of those functions with explicitly defined diagnostics and an interface for reviewing results. Extension to real lending operations would require institution-specific data governance, security controls, external model validation, and human evaluation.

Bibliography

[1] D. J. Hand and W. E. Henley, "Statistical classification methods in consumer credit scoring: A review," *Journal of the Royal Statistical Society: Series A (Statistics in Society)*, vol. 160, no. 3, pp. 523–541, 1997.

[2] S. Lessmann, B. Baesens, H.-V. Seow, and L. C. Thomas, "Benchmarking state-of-the-art classification algorithms for credit scoring: An update of research," *European Journal of Operational Research*, vol. 247, no. 1, pp. 124–136, 2015.

[3] C.-K. Yeh, B. Kim, S. O. Arik, C.-L. Li, and T. Pfister, "On the (in)fidelity and sensitivity of explanations," in *Advances in Neural Information Processing Systems*, vol. 32, 2019.

[4] D. Alvarez-Melis and T. S. Jaakkola, "On the robustness of interpretability methods," in *arXiv preprint arXiv:1806.08049*, 2018. [Online]. Available: https://arxiv.org/abs/1806.08049

[5] E. Tabassi, "Artificial intelligence risk management framework (AI RMF 1.0)," National Institute of Standards and Technology, Tech. Rep. NIST AI 100-1, 2023.

[6] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *The Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.

[7] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.

[8] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems*, vol. 30, 2017, pp. 4765–4774.

[9] M. T. Ribeiro, S. Singh, and C. Guestrin, "Why should I trust you? explaining the predictions of any classifier," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 1135–1144.

[10] M. Hardt, E. Price, and N. Srebro, "Equality of opportunity in supervised learning," in *Advances in Neural Information Processing Systems*, vol. 29, 2016.

[11] S. Wachter, B. Mittelstadt, and C. Russell, "Counterfactual explanations without opening the black box: Automated decisions and the GDPR," *Harvard Journal of Law & Technology*, vol. 31, no. 2, pp. 841–887, 2018.